

ByteCheckpoint: 面向大型基础模型开发的统一 Checkpoint 系统

Borui Wan^{1,*} Mingji Han^{2,*} Yiyao Sheng² Yanghua Peng² Haibin Lin² Mofan Zhang²
Zhichao Lai² Menghan Yu² Junda Zhang² Zuquan Song² Xin Liu² Chuan Wu¹

¹The University of Hong Kong ²ByteDance

Abstract

在大型基础模型 (Large Foundation Models, LFM) 的开发过程中, 为了在各类故障或 GPU 资源与 parallelism 配置变化后恢复训练, checkpointing 保存训练状态至关重要。此外, 保存的 checkpoints 会被分发到 evaluation 任务或不同训练阶段之间转移 (例如从 pre-training 到 post-training)。这些场景都需要将分布式 checkpoints 从一种 parallelism 重新分片 (resharding) 到另一种。在生产环境中, 不同 LFM 会依据模型规模与训练规模采用不同的 training framework 和 storage backend。因此, 需要一个高性能的 checkpointing 系统, 以在 LFM 全生命周期内实现大规模高效的 checkpoint 管理。

我们提出 ByteCheckpoint, 一个面向大规模 LFM 训练的工业级 checkpointing 系统。ByteCheckpoint 具备: 与 parallelism 无关的 checkpoint 表示, 可在加载时 (load-time) 高效 resharding; 面向多 training framework 与多 storage backend 的通用保存/加载 workflow; 保障高 I/O 效率与可扩展性的 full-stack 优化; 以及用于大规模性能分析与瓶颈定位的监控工具套件。与现有开源 checkpointing 系统 [51, 57] 相比, ByteCheckpoint 显著降低运行时 checkpoint stall, 平均减少 54.20 $\times$ 。在保存与加载时间上, ByteCheckpoint 最高分别提升 9.96 $\times$ 与 8.80 $\times$ 。

1 引言

语言 [5, 13, 33, 43]、视觉 [40, 44]、音频 [58] 等模态的 Large Foundation Models (LFMs) 正在重塑当今 AI 生态, 催生了诸如下游应用: 对话式代理 [48, 55]、编码助手 [50]、绘画工具 [52] 以及视频 [54] 生成器。

与传统深度学习模型训练不同, LFM 训练显著更为复杂。其包含多个训练阶段, 包括 pre-training [9, 25] 与 post-training [38, 46, 66, 69]。此外, 这些阶段还会集成 evaluation 任务以有效评估模型质量。除了流程复杂, LFM 的开发还极其资源密集且耗时, 主要源于巨大的

模型规模与训练数据量 (例如, DeepSeek-V3 拥有 6710 亿总参数, 并在 14.8 万亿 tokens 上完成 pre-training) [13, 33]。LFM 训练规模甚至可达到 12,288 GPUs [25]。

作为保存训练状态的基础技术, checkpointing 会对这些状态进行快照并存入持久化存储, 以便训练恢复。此外, 在 LFM 训练过程中, checkpoints 还会被用于并行的 evaluation 任务以持续评估模型质量。另一类应用是 将这些快照从 pre-training 分发到下游 post-training 任务, 例如 Supervised Fine-Tuning (SFT) 或强化学习。复杂的开发流水线、巨大的训练规模与漫长的训练周期, 使得设计一个高效的 checkpointing 系统面临显著挑战。

第一, 真实世界的 LFM 开发生命周期内需要高效且统一的 checkpoint 管理。训练过程中经常需要进行 checkpoint resharding。该过程会将已保存的分布式 checkpoints 转换到与保存时不同的新 parallelism 配置, 以便正确加载。在 pre-training 中, parallelism 的变化可能来自故障机器下线、可用 GPU 配额波动 [29, 71], 或训练配置 (如 context length) 与系统优化技术 [25] (如 kernel fusion、计算与通信重叠) 调整。跨训练阶段与任务时, parallelism 会随资源规模与数据集规模而变化。高效的 checkpoint resharding 可最小化额外开销并最大化端到端 Effective Training Time Ratio (ETTR, 定义为有效训练时间与作业 wallclock 时间之比) [27]。此外, 在工业 AI 平台上, 不同内部用户或云客户发起的 LFM 训练会选择不同训练框架 (如 Megatron-LM [47]、PyTorch FSDP [72]、DDP [31] 以及其他 [62]) 并按任务特性与偏好选择 storage backend (如本地磁盘、HDFS、NAS) 保存 checkpoints。为适配这些临时组合而定制 checkpoint 模块与流程, 会显著增加系统开发与维护成本。因此, 为不同训练框架与 storage backend 提供通用 workflow 至关重要。

第二, 需要高效且可扩展的 I/O 性能。checkpointing 系统在将分布式 checkpoints 写入持久化存储、以及在不同场景下加载回训练时, 都涉及大量 I/O 操作。为了缩短 I/O 阻塞时间并快速持久化训练状态, 必须加速这些 I/O 工作负载。同时, 现代 LFM 训练常在大规模运行, 因此 checkpointing 系统的可扩展性同样关键。

现有 checkpointing 系统 [14, 19, 35, 65] 假设保存与加载时 parallelism 一致, 无法满足 checkpoint reshard-

* 同等贡献。已被 NSDI'25 接收。代码: [ByteCheckpoint](#)。

ing 的需求。尽管开源社区已有一些支持 resharding 的系统,但仍存在诸多限制。有的采用低效的离线 resharding [32],有的仅支持特定训练框架与 storage backend [51, 57]。此外,缺乏定制化优化与工业级部署经验,它们通常 I/O 性能不足,且难以在真实生产中支撑大规模 LFM 训练。

本文介绍 ByteCheckpoint 的设计、实现与部署经验,这是一套为 LFM 开发打造的 checkpointing 系统。ByteCheckpoint 采用统一架构 (Sec. 3.1), 提供自动 *resharding-on-loading* 的分布式 checkpoint 机制 (即 load-time checkpoint resharding), 并支持多种训练框架与 storage backend。同时,系统集成了 full-stack I/O 性能与可扩展性优化。

▷ ByteCheckpoint 的 checkpoint 表示与训练时采用的具体 parallelism 解耦 (Sec. 3.2), 从而支持高效的 load-time checkpoint resharding。在模型与优化器状态表示上,我们将每个 tensor shard 的 metadata 与数值数据分离,并将全部 metadata 汇总到一个全局文件。单个 tensor shard 的 metadata 包含其运行时、位置与存储相关信息。对于 dataloader 状态表示,我们将其划分为 *replicated* 与 *sharded* 两类: *sharded* 状态保存为独立文件,而 *replicated* 状态仅由 global rank 0 的训练 worker 保存。

▷ ByteCheckpoint 提供面向不同训练框架与多种 storage backend 的通用 workflow (Sec. 3.3)。它为每个框架定制 planner 生成统一的保存/加载 plan,并将这些 plan 交由与框架、storage backend 无关的 engine 执行 I/O。利用架构层面的隔离,ByteCheckpoint 能为不同框架与 storage backend 的用户执行相同的保存/加载流程。

▷ ByteCheckpoint 实现了多项优化 (Sec. 4) 以提升 I/O 性能,包括负载均衡与零冗余 plan 生成、全异步执行 pipeline 以及高效的 irregular tensor 处理。我们还分享了在优化 storage system 以支持海量 I/O 请求、优化集体通信以保证稳定性、以及设计监控与可视化工具以进行性能分析与瓶颈定位方面的工程经验 (Sec. 5)。该 full-stack 方法使 ByteCheckpoint 能在 8,960 GPUs 上训练 405B 规模的 LFM 仍保持高效。

ByteCheckpoint 已部署在拥有数万 GPU 的工业 AI 平台上,用于多种 LFM 训练任务,包括语言模型的 pre-training 与 post-training、多模态理解,以及生成类模型 (e.g. 用于视频生成)。ByteCheckpoint 相较现有开源 checkpointing 系统 (如 PyTorch DCP [57] 和 Megatron Distributed Checkpoint [51]) 具有显著优势: checkpoint stall 减少 $12.13\times$ 到 $161.50\times$, 端到端保存与加载流程平均分别快 $6.05\times$ 与 $3.88\times$ 。

2 背景与动机

2.1 LFM 开发

生产流水线。如图 1 所示, LFM 的开发包含 pre-training 与 post-training 阶段。在初始 pre-training 阶段, LFM 在来自多源的大规模数据上迭代训练,以吸收世界知

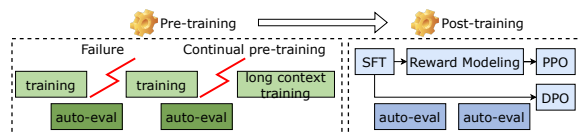


Figure 1: LFM 训练流水线概览。

识。随后,通过持续 pre-training 进一步增强基础模型能力。例如,大语言模型 (LLM) 的 pre-training 通常会进行长上下文持续训练,以逐步提升模型支持的 context length。post-training 用于对齐人类反馈或提升模型推理能力 [18, 56]。在 fine-tuning 中会使用多种任务特定的标注数据集 (e.g., 多语种、代码、数学、推理等), 随后进行强化学习, 通常包括 reward modeling 与 Proximal Policy Optimization (PPO) [45], 或直接向 Direct Preference Optimization (DPO) [41]。由于这些数据集规模较小, post-training 通常使用更少的 GPUs。在这些阶段中,还会周期性触发自动评测 [14, 22], 拉取中间 checkpoints 并通过多维度指标评估模型质量。

Checkpointing. LFM 训练任务的训练状态包括 GPU 与 CPU 状态。GPU 状态是 LFM 的可学习参数以及优化器信息 (例如 Adam [26] 中模型的 float32 精度副本及其动量与方差)。在最先进的大规模并行训练 [28] 中,这些状态会被分片并分布在多张 GPU 上。CPU 状态包括 dataloader 模块、随机数生成器 (RNG) 状态、全局训练步数以及学习率调度器等,均存放在 CPU 内存。我们的 dataloader 模块包含 token buffer, 用于缓存来自数据源的不同长度样本; 当累计 token 数达到 context window size [13, 25, 68, 70] 时, dataloader 会将缓存样本组装成一个 batch (micro-batch)。由于 GPU 与 CPU 内存具备易失性,这些训练状态需要周期性保存到持久化存储,以容忍故障并支撑 evaluation 任务。

Storage backends. 在生产环境中,训练通常使用独立的分布式文件系统 (如 Llama 3.1 训练使用的 Tectonic [39] [13]) 存储 checkpoints [14, 20]。鉴于训练中不可避免的硬件故障与软件缺陷 [13, 22, 25], 必须在不同训练步上保存 checkpoints 以保障训练安全。分布式文件系统 (如 HDFS、NAS) 提供充足的存储容量,以容纳大型模型的多个 checkpoints。

2.2 Checkpoint Resharding 场景

在 LFM 开发生命周期中, parallelism 在不同场景下发生变化,因此 checkpoint resharding 持续存在 (图 2): (1) 训练恢复。LFM 训练任务的 GPU 配额可能因故障机器移除 [22, 25]、新增可用机器,或使用潮汐资源 [12, 29, 67, 71] 而发生变化。为了最大化资源利用率,往往需要据此调整训练 parallelism。此外,在大规模 pre-training 初期,工程师通常需要探索不同模型配置与优化技术 [25], 因为小规模 profile 或仿真结论未必适用于大规模训练。在这一阶段, parallelism 配置频繁调整,导致训练反复恢复。此外,长上下文训练会改变 context length,也会引发 GPU 配额或 parallelism 调整。如图 2 所示示例,最初保存了 8 个分布式 checkpoint

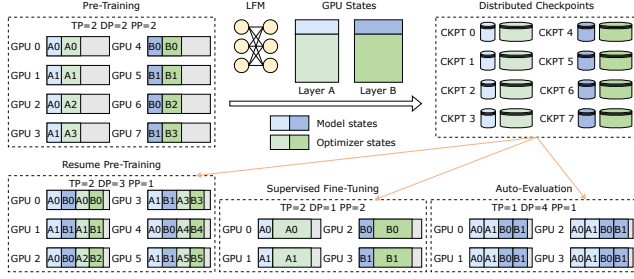


Figure 2: LFM 训练中的 checkpoint resharding 场景。为便于展示，图中仅绘制 GPU 状态。

文件，恢复时加载到 6 个训练 worker。

(2) 跨阶段切换。进入 post-training 阶段时，因后续训练数据规模更小，参与训练的 GPU 数量通常降低。pre-training 保存的 checkpoints 需要 resharding 以适配各类 post-training 任务的具体工作负载。如图 2 所示，post-training 的 fine-tuning 任务仅使用 4 GPUs，因此需要将 checkpoints 进行相应 resharding。

(3) 评测。两个阶段的 evaluation 任务都需要加载模型 checkpoints 并在独立资源上进行推理；其 parallelism 需与评测任务分配的 GPU 资源匹配。图 2 展示了一个 4-GPU 的评测任务从 pre-training checkpoints 进行 resharding 的场景。

我们统计了平台最近六个月（训练文本、图像与视频生成模型）的 resharding 需求：pre-training 恢复场景 1,870 次，跨阶段重配置 13,080 次，评测任务 19,844 次。

2.3 LFM Checkpointing 需求

Load-time checkpoint resharding. 作为 LFM 开发生命周期中的关键步骤，checkpoint resharding 的效率决定了额外开销的大小。在我们的 AI 平台上，过去的常见做法是编写离线 resharding 脚本，并在出现新 resharding 场景时调整脚本。该方法低效且人力成本高（详见附录 A）。除了开发成本，离线脚本运行还会浪费大量 GPU 时间与资源。表 1 给出了不同场景下离线 resharding 作业的开销。在训练恢复或评测开始之前，必须先提交独立作业执行 resharding：从存储系统下载 checkpoints，按指定 parallelism 进行 resharding，再上传到存储系统。目标训练或评测作业在 resharding 作业完成前无法启动，导致等待时间延长。此外，离线脚本生成的 checkpoints 与特定 parallelism 绑定，难以复用，增加了存储开销。

相比之下，load-time checkpoint resharding [57]（也称在线 resharding）更适合 LFM 训练。它在加载阶段自动识别并从现有分布式 checkpoints 中提取所需数据。要实现这一点，checkpoint 的存储表示必须与具体 parallelism 解耦。

多训练框架与多 storage backend. 我们的 AI 平台上使用了多种训练框架，如 Megatron-LM [47]、DDP [31]、FSDP [72]、veScale [25,62] 等。表 2 基于六个月的轨迹分析给出了平台上最常用的三类训练框架。用户通常使用

Table 1: 不同场景下离线 resharding 作业的平均完成时间。

Training Resumption	Cross-Stage Transition	Evaluation
1870.38s	650.34s	593.21s

Table 2: 平台上使用最广泛的三类训练框架

Framework	Pre-training	Post-training	Average #GPUs Per Job
Megatron-LM	13727	68621	301
FSDP	16842	†	25
DDP	25393	†	6

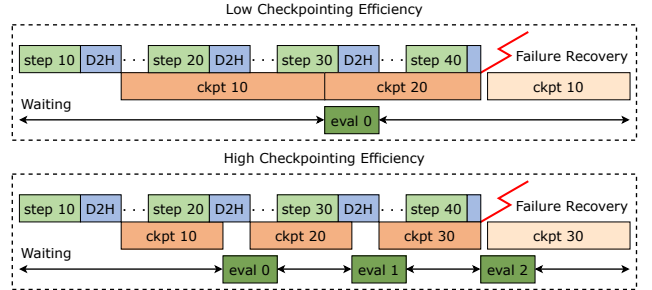


Figure 3: Checkpointing 效率对故障恢复与评测任务的影响。D2H 表示 Device-to-Host 拷贝。

Megatron-LM [47] 训练大型语言基础模型。FSDP [72] 常用于 text-to-video 或 text-to-speech 模型训练，而 DDP [31] 则多用于多模态基础模型中的图像编码器训练或日常算法测试。此外，用户还可根据场景选择不同 storage backend 保存 checkpoints，从调试时的本地磁盘到正式训练时的 HDFS 或 NAS。

每种训练框架都自带 checkpoint 模块、文件格式与保存/加载逻辑的临时实现。然而这些模块缺乏生产级能力，如 load-time resharding、异步 checkpointing、以及对远程持久化存储的支持。针对每个框架单独补齐这些能力并实现优化，意味着重复的工程成本，也造成跨框架/存储 backend 的接口不一致，增加代码复杂度。同时，不同框架的 checkpoint 文件格式差异也增加了跨阶段 checkpoint 转移与评测/推理部署的复杂度。因此，需要为不同训练框架与 storage backend 提供通用 workflow。

高效且可扩展的 I/O 性能. 主流 LFM 模型规模巨大，可达数千亿参数 [13]。因此训练状态大小也显著增加，带来 checkpoint 保存与加载的巨大开销。基于我们对历史 LFM 训练任务的分析，4096 GPUs 训练的 GPT-175B 模型，其分布式 checkpoints 写入 HDFS 的平均端到端时间可达 200 秒，远超单次训练迭代时间。尽管采用异步 checkpointing [25,34,35,64] 能将一部分开销移出训练关键路径，但端到端 checkpointing 的加速仍至关重要，可在不可避免的频繁故障 [25] 中减少训练进度损失 [65]。如图 3 所示，即使 checkpointing 与训练并行进行，若能更快完成，就能在故障前保存更多中间 checkpoints，从而恢复到更近的状态并提升 ETTR。此外，训练过程中会触发 evaluation 任务并周期性拉取中

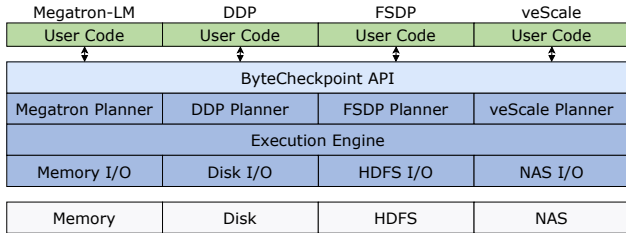


Figure 4: ByteCheckpoint 的架构。

间 checkpoints, checkpointing 越快, 评测准备阻塞越少。

在保持高 I/O 性能的同时进行规模扩展并不容易。一些在小规模下难以发现的次优或高风险设计, 可能在大规模训练中导致严重性能瓶颈甚至灾难性作业失败。例如, 训练集群到存储系统 (如 HDFS) 的海量读写请求可能压垮 master 节点, 导致文件元数据操作延迟。此外, 朴素的集体通信实现 (如完整性检查 barrier) 会引入显著的初始化与同步开销, 甚至触发通信超时, 从而导致训练失败。随着规模增长, 在跨多机的训练与 I/O worker 间高效分析性能与检测错误也愈加困难。

现有 checkpointing 系统 (如 CheckFreq [35]、CheckN-Run [14]、Gemini [65]) 都假设 parallelism 不变, 无法满足 resharding 需求。DCP [57] 与 MCP [57] 虽具备 resharding 能力, 但在并行策略/框架支持、I/O 性能与可扩展性方面仍受限。我们在附录 F 对相关工作进行了更全面讨论。在 ByteCheckpoint 中, 我们设计了与 parallelism 解耦的存储表示来支持高效 load-time resharding, 提出通用保存/加载 workflow 以支持多框架与多 storage backend, 并集成 full-stack 优化以提升 I/O 性能, 同时分享了在真实 LFM 训练中扩展 checkpointing 系统的经验。

3 系统设计

基于上述观察, 我们按照以下关键原则设计 ByteCheckpoint:

- (1) 解耦。checkpoint 表示与具体运行时 parallelism 保持独立。训练框架与 storage backend 的接口与核心执行引擎分离, 确保良好的可扩展性。
- (2) 易用性。API 应当简洁, 便于 AI 研究人员与工程师无缝集成到代码与运行环境中。

3.1 统一架构

ByteCheckpoint 的架构如图 4 所示。各组件如下:

API. ByteCheckpoint 的 API (`bytecheckpoint.save` 与 `bytecheckpoint.load`) 为不同训练框架的用户提供统一入口。例如保存 checkpoint 时, 用户准备好训练状态、checkpoint 路径、框架名称与性能相关选项, 然后调用 `bytecheckpoint.save`。该高层入口屏蔽了底层复杂度, 例如分片规范、保存/reshard plan 生成与 I/O 操作。

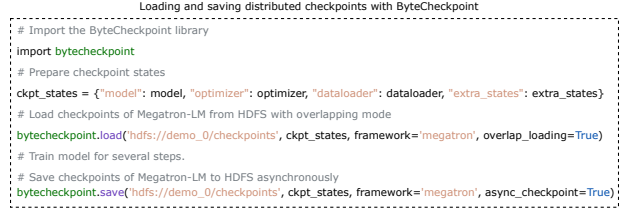


Figure 5: ByteCheckpoint API 的使用示例。

Planner. Planner 是训练框架的接口层。它从 API 层接收参数 (训练状态、checkpoint 路径等), 根据 worker 的 rank 与框架特定的分片规范 (如 *Megatron ShardedTensor* 或 *FSDP DTensor*) 为每个 tensor shard 创建 *ShardMeta* (Sec. 3.2), 并决定每个 worker 需要保存/加载的 tensor 与其他状态。每个 worker 先生成本地 plan, 再协同生成全局 plan。我们为每个训练框架实现定制化 planner, 以抽取这些规范信息并生成 plans。

Execution Engine. 引擎运行在每个训练 worker 上, 当调用相应 API 时执行 Planner 生成的保存/加载 plans。它解析 checkpoint 路径以确定对应的 storage backend, 并与 Storage I/O 层交互以执行 plans 中的 I/O 任务。

Storage I/O. 与 Planner 层类似, Storage I/O 层封装不同 storage backend, 并管理后端特定的读写操作与优化。Engine 与 Storage I/O 层之间保持统一接口, 便于无缝集成新的 storage backend。ByteCheckpoint 支持多种 storage 选项, 包括 in-memory checkpoint 存储 [65]、本地磁盘存储与远程存储系统。

图 5 展示了 ByteCheckpoint 的典型使用方式。首先, 用户定义一个字典, 指定需要保存或加载的状态 (模型、优化器、dataloader 以及其他状态)。在训练恢复、阶段切换或 evaluation 开始时调用 `bytecheckpoint.load` 以加载已保存的 checkpoints。当 parallelism 发生变化时, checkpoint resharding 会在加载过程中自动完成。训练过程中, `bytecheckpoint.save` 会周期性调用以保存 checkpoints。

3.2 解耦的 Checkpoint 表示

为了实现与 parallelism 无关的 checkpointing 并支持自动 load-time resharding, 我们提出分布式训练中模型与优化器张量的形式化表示规范。该设计与 *PyTorch Distributed* 库 [31] 中的 *Distributed Tensor* 与 *Checkpoint* 概念保持一致。

模型与优化器状态。每个 tensor 由唯一的“fully qualified name” (FQN) 标识, 并具有全局形状, 表示分片前的原始形状。Tensor 可以在不同 rank (训练 worker) 间分片或复制。对于分片 tensor, 每个 worker 持有的 shard 由三类信息决定: 该 tensor 所采用的 parallelism (如 tensor parallelism)、分片维度、以及在对应 parallelism group 内的 group rank。基于 tensor 的 FQN、全局形状与分片规范, ByteCheckpoint 为每个 rank 创建 tensor shard 的 metadata (*ShardMeta*), 用于在 checkpoint 中表示该 shard, 且与具体 parallelism 无关。具体而言, *ShardMeta*

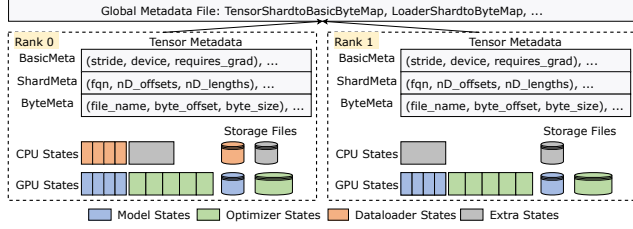


Figure 6: ByteCheckpoint 中的 checkpoint 表示。该示例采用 pipeline parallelism。

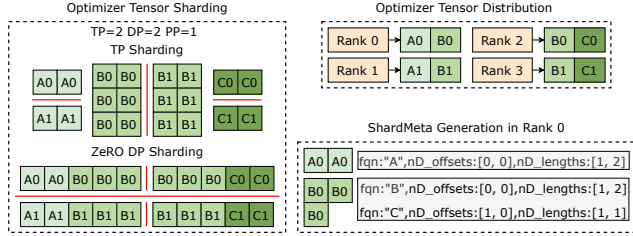


Figure 7: 优化器状态中 irregular tensor 的 ShardMeta。

是一个索引元组 ($fqn, nD_offsets, nD_lengths$), 其中 $nD_offsets$ 与 $nD_lengths$ 表示该 shard 在原始多维形状上的偏移与长度。

Dataloader 与其他状态。Dataloader 状态可划分为两类: *replicated* 状态与 *sharded* 状态。replicated 状态包含数据读取 worker 数、数据源路径与采样比例等信息, 且在不同 rank 的所有 I/O worker (子进程) 之间一致。sharded 状态在每个 I/O worker 间不同, 包括 token buffer 与各数据源的读取偏移。在 ByteCheckpoint 中, sharded 状态保存为独立文件, 而 replicated 状态仅由 rank 0 的 I/O worker 保存。该策略减少了需保存的 dataloader 状态总体大小, 并在 parallelism 变化时便于 dataloader resharding, 因为将数据状态拆分为独立文件能简化合并与重分配过程。对于 RNG 等其他额外状态, 我们将其打包并序列化为一个紧凑的字节对象后写入存储。

Checkpoint 表示。图 6 展示了 ByteCheckpoint 的 checkpoint 表示。分布式 checkpoints 由一个全局 *meta-data* 文件与多个存储文件组成。每个 rank 生成三类文件: 模型状态文件、优化器状态文件与额外状态文件。dataloader 状态文件仅由除 DP 之外各 parallelism 度的 rank 为 0 的训练 worker 生成。对于模型与优化器状态中的 tensor shards, 其 *metadata* 由三部分组成: *BasicMeta* (记录单个 tensor shard 的必要信息, 如 stride 与 device, 用于恢复运行时状态); *ShardMeta* (记录 shard 在完整 tensor 中的位置); *ByteMeta* (记录该 shard 在存储文件中的字节起始偏移与长度)。所有 tensor metadata 被汇总到全局 metadata 文件中, 并基于每个 shard 的 *ShardMeta*、*BasicMeta* 与 *ByteMeta* 建立映射 *TensorShardtoBasicByteMap*, 确保准确取回数据。全局 metadata 文件还包含 *LoaderShardtoByteMap*, 记录每个 dataloader 中 sharded 状态的文件索引信息。所有存储文件与全局 metadata 文件都保存在 checkpoint 路径指定的 storage backend 中。

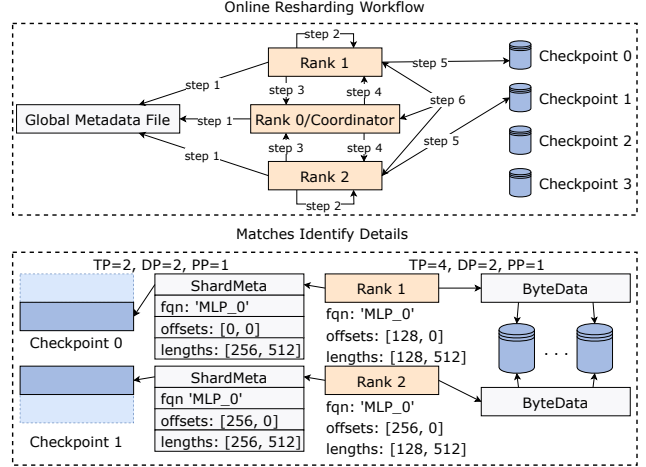


Figure 8: Load-time tensor resharding 工作流。Checkpoint k 表示由先前运行的 worker k 保存的 checkpoint。

Irregular tensor 分解。我们将 irregular tensor 定义为: 在分片前被扁平化后, 无法恢复回原始维度的 tensor。这类 tensor 通常出现在 Zero Redundancy Optimizer (ZeRO) [42] 的实现中, 例如 Megatron-LM ZeRO2 与 FSDP ZeRO3 [72]。在这些实现中, DP 组内的优化器状态会被扁平化、拼接并分片, 导致得到的 1D tensor 切片无法直接用 n 维形状与偏移描述。图 7 展示了 irregular tensor 分片示例: 与张量 A 和 C 不同, 原始形状为 (3, 2) 的张量 B 被均匀分成两个 shard, 每个 shard 含 3 个元素, 无法直接表示为二维形状及其偏移。一种直观方案是在保存 checkpoint 前将所有 tensor shards 合并为完整 tensor, 以便生成 *ShardMeta*。例如, 为消除 DCP [57] 中潜在的 irregular tensor, FSDP 会对每个 shard 执行同步 all-gather, 并与 D2H 拷贝交错进行, 无论该 shard 是否 irregular。但这种做法带来显著通信开销, 并需要频繁 GPU/CPU 同步, 严重影响效率。

为降低合并开销, ByteCheckpoint 采用 tensor 分解策略处理 irregular tensor shards。具体来说, ByteCheckpoint 将一个 irregular tensor 分解为多个 regular tensor, 并用多个索引元组进行表示。例如, 图 7 中 rank 0 上张量 B 的 irregular shard 含 3 个元素, ByteCheckpoint 将其分解为两个 regular tensor, 可直接由 $nD_offsets$ 与 $nD_lengths$ 表示。因此, ByteCheckpoint 使用多个 *ShardMeta* 条目来表示单个 irregular tensor shard。尽管分解会稍微增大 metadata 体积并增加加载过程 (重建目标 tensor 可能需要查找多个小片段), 但它避免了保存阶段昂贵的 shard 通信, 同时不引入额外阻塞。

3.3 Checkpoint Resharding 工作流

ByteCheckpoint 实现了统一的保存/加载工作流, 并在加载过程中自动 resharding。借助 Engine、Planner 与 Storage I/O 层之间的解耦, 我们可在不同训练框架与 storage backend 上执行一致的保存/加载步骤。以下以 tensor resharding 为例 (图 8), 说明我们的 checkpoint 表示如何支持灵活的 load-time resharding。对于不涉及

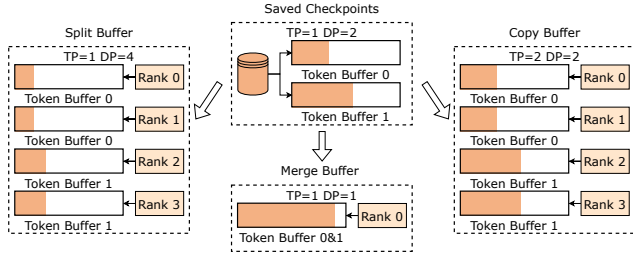


Figure 9: Dataloader resharding 示例。并行配置变化时，对 cumulative token buffer 等 sharded 状态进行 resharding。为清晰起见，图中未绘制数据读取偏移。

resharding 的保存与加载，流程类似。

Step 1. 每个 rank 调用 `bytecheckpoint.load()`，指定 checkpoint 路径以及待恢复的模型/优化器等状态。所有 rank 从路径加载全局 metadata 文件。

Step 2. 对于模型/优化器中的每个 tensor shard，每个 rank 查询全局 metadata 中的 `TensorShardToBasicByteMap`，识别已保存 shards 与新分片规范之间的匹配片段。识别完成后，planner 构建本地加载 plan，其中包含该 shard 对应的 `BasicMeta` 与 `ByteMeta`。图 8 底部示意了这一匹配机制。

Step 3. 协调者 planner（通常位于 rank 0）发起 gather 操作汇总各 rank 的加载 plan，并应用冗余消除优化（Sec. 4.1）以分配加载工作量、缩短完成时间。

Step 4. 协调者发起 scatter 操作下发最终加载 plan，各 rank 接收其最终 plan。

Step 5. 每个 rank 的执行引擎根据 checkpoint 路径选择 Storage I/O 层中的 backend wrapper，并执行加载 pipeline（Sec. 4.2）。

Step 6. 加载 pipeline 完成后，各 rank 使用优化后的异步 collective barrier 原语确保分布式加载的原子性。更多细节见附录 B。

Dataloader resharding. 类似图 8 的 tensor resharding，ByteCheckpoint 通过查询 `LoaderShardtoByteMap` 实现 dataloader resharding。图 9 展示了 dataloader resharding 的示意。当 parallelism 配置变化时，dataloader checkpoints 中的公共条目可直接加载，而 token buffer 与数据读取偏移等独有条目需要 resharding。具体而言，当 DP 度保持不变而其他 parallelism 度改变（图 9 中 TP 改变）时，需要将 token buffer 复制到目标 worker 以保证 bitwise 正确的恢复；当 DP 度变化时，token buffer 需要相应拆分或合并，以确保恢复后的 dataloader 不会丢失缓存数据、也不会重复训练已采样的数据。得益于 dataloader 模块的分拆存储策略，ByteCheckpoint 能精确识别需要 resharding 的独有条目并高效处理。

4 性能优化

下面介绍 ByteCheckpoint 的性能优化技术，重点在于降低 checkpoint 保存与加载相关开销。

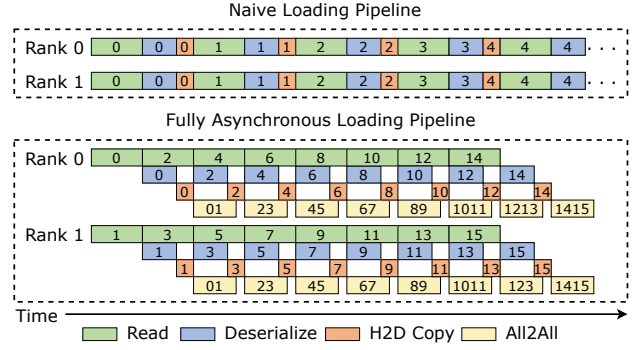


Figure 10: ByteCheckpoint 加载 pipeline 与朴素实现的对比。

4.1 优化的 Plan 生成

保存负载均衡。在采用 data parallelism（DP）的训练场景中，模型状态在所有 DP 组间复制，导致模型状态重复。现有 checkpointing 系统 [51, 57] 通常指定第一个 DP 组保存全部模型状态，但这会造成负载不均，使第一组成为 straggler。为解决该问题，我们在规划阶段引入负载均衡的去重机制，采用 Worst-Fit 算法：协调者根据每个 tensor shard 的大小分配保存任务，总是将当前 shard 分配给累计 shard 大小最小的 rank。这样可在各 rank 之间更均衡地分配保存负载，提升保存效率。消除冗余加载。当加载到包含 DP 的 parallelism 配置时，ByteCheckpoint 通过消除 DP 组间的重复读取来优化加载过程，将存储读取与 tensor 传输结合起来。关键观察是：在文件读取期间，可利用空闲的跨 GPU 带宽同步把已加载 tensor 传给其他 GPU。如图 10 所示，在规划阶段我们将读取任务在 DP 组内均匀分配，避免重复读取；随后启动 I/O 线程读取分配的 tensor shards。与此同时，在主线程中，CPU 内存中的 shards 先拷贝到 GPU，再通过 all-to-all 等集体通信传给需要的其他 worker。

Plan 与 metadata 缓存。在大规模训练中，规划过程会带来显著的通信开销，尤其是在高频 checkpoint 保存时。例如，为一个分布在 8960 GPUs 上的 405B transformer 模型规划保存流程需要 62 秒。然而我们观察到，尽管 save plan 与全局 metadata 与具体 parallelism 绑定，但在同一次训练会话中保持不变。因此可以通过缓存减少后续 checkpoint 操作的规划时间与开销。我们引入 plan 与 metadata 缓存，将规划开销变为一次性成本。首次生成后，save plan 与全局 metadata 可在后续复用，避免重复规划。

4.2 全异步的 Engine Pipeline

ByteCheckpoint 的 Engine 通过流水线化执行优化 checkpoint 保存与加载（resharding）。以加载为例（图 10），我们将每个 tensor shard 的文件读取、反序列化、Host-to-Device（H2D）拷贝以及 GPU 间通信进行流水线化，从而提升效率。read 操作会从存储系

统下载包含所需 tensor 的文件，并放入共享内存（如 `dev/shm` 目录）；`deserialize` 将共享内存中的内容反序列化为 tensor。ByteCheckpoint 使用多线程并行进行文件下载与反序列化。`H2D copy` 将 tensor 从 CPU 内存传到 GPU，而 `All2All` 在 DP 组内进行 tensor 传输。

对于保存，我们实现了对称的全异步 pipeline，包含 D2H 拷贝、序列化与文件上传。为降低 D2H 对训练的影响，我们使用固定页（pinned）CPU 内存池，并结合 Ping-Pong 缓冲机制加速该操作。此外，我们运行多个并行进程进行序列化并将文件写入共享内存。上传线程在 dump 完成后自动检测并启动上传。

5 大规模 Checkpointing

本节深入介绍在大规模 LFM 训练中优化 checkpointing 的方法。

5.1 存储支持

高吞吐、可扩展的存储系统。HDFS 是 ByteCheckpoint 的主要 storage backend，我们针对大规模 checkpointing 实现了多项吞吐与可扩展性优化。我们将 HDFS 的全部组件（NameNode、DataNode 与 SDK [15]）重写为 C++ 版本，相较原 Java 实现性能翻倍。此外，我们在架构中引入新组件 NNProxy [1]。NNProxy 是 HDFS NameNode 的无状态 RPC 代理，可支持大规模 NameNode 联邦部署并保持低查询延迟。其设计缓解了 NameNode 元数据请求的 QPS 瓶颈，该问题在分布式 checkpoints 大量产生时尤为突出。NNProxy 还提供认证、限流与元数据查询缓存等能力，使得生产环境中 checkpoints 的管理更细粒度。经过优化的 HDFS 可提供数百 PB 容量、10 TB/s 读写带宽以及约 100,000 的元数据 QPS。

Checkpoint 冷却策略。我们构建了 SSD+HDD 的双层冷热存储架构。关键观察是：新生成的 checkpoint 文件通常会在短时间内被 evaluation 任务读取；若训练过程未出现 loss spike 等异常，其访问频率会显著下降。但出于可追溯性考虑，所有 checkpoint 文件仍需保留。为在保证可用性的同时高效利用存储空间，我们设计了数据冷却（cool-down）机制，将数据从 SSD 迁移至 HDD，使高性能热存储始终有足够空间。具体做法是：根据文件最近修改时间，将超过保留阈值的文件进行冷却，并通过纯元数据操作将原路径映射到 HDD 新位置。该策略在 HDFS 目录级别生效，并保持原访问路径不变，用户体验无缝。

5.2 集体通信

在 ByteCheckpoint 的工作流（Sec. 3.3）中，集体通信（如 scatter、gather、barrier）是规划与 checkpoint 完整性保障的关键环节。我们最初使用 NCCL [53] 作为通信后端执行协调者的 gather/scatter。但在将 pre-training 扩展到 8960 GPUs 时发现：NCCL 在规划阶段需要耗费较长时间惰性创建通信通道与分配 GPU 内存；在某些情况下甚至会无响应或触发 CUDA OOM（因为

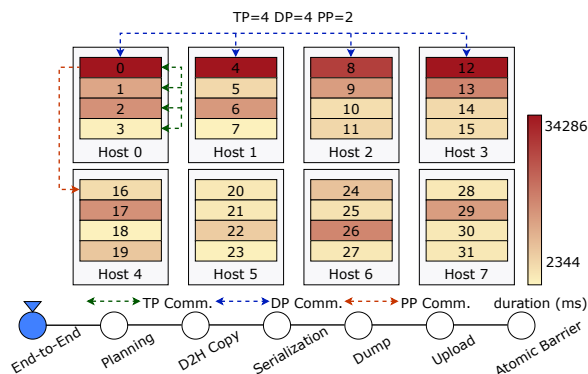


Figure 11: 采用 Megatron-LM 的 32-GPU 3D 并行训练任务中，端到端 checkpoint 保存时间热力图。颜色表示某一 rank 在选定阶段的耗时。

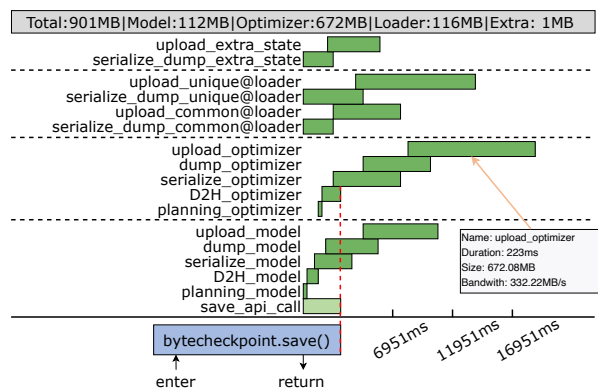


Figure 12: rank 0 上 checkpoint 保存的时间分解。

scatter/gather 需要与每个 GPU 建立 P2P 连接)。这些 GPU OOM 与长初始化时间在小规模测试中并不明显，但在大规模下成为显著问题。

为提升规划阶段通信稳定性，我们使用 gRPC [17] 重写该流程，避免规划期间占用 GPU 内存。然而，当训练规模扩展到数万 GPUs 时，集中式 gather/scatter 仍对协调者造成巨大负载，引发通信失败。因此我们进一步采用树形分层通信拓扑提升稳定性。同一台机器上的训练 workers 组成一级子树，以本地 rank 0 作为根；跨机器通信时，将多台机器迭代分组，每组内全局 rank 最小的 worker 为根，直至形成汇聚到全局根（协调者）的层级结构。在大规模 3D 并行训练中，TP-DP-PP 通信树可自然形成，避免额外连接。

5.3 监控与分析

训练过程中，ByteCheckpoint 持续收集关键性能指标并进行可视化，用于实时性能监控与分析。这使我们能够快速检测 checkpointing 问题，例如低读写吞吐、straggler，以及上传失败与重试等。

数据采集。我们基于 Python 的 context manager 与 decorator 语法设计了易用的指标系统，可灵活监控关键流程。系统自动采集每个操作的耗时与 I/O 大小，并

记录 worker rank、文件路径与当前 step 等元数据。所有指标通过后台消息队列传输至远端数据库。

可视化。ByteCheckpoint 提供两类视图：一是不同 checkpoint 阶段、不同 rank 的整体拓扑性能概览；二是任意单个 rank 的详细耗时分解。图 11 展示了 3D 并行拓扑上的 checkpoint 保存时间热力图，其中 rank 0、4、8、12 由于包含 dataloader 状态，保存时间最长。用户还能选择其他细粒度阶段（如 planning 或 D2H）进行分析。这些可视化有助于快速定位系统开发与生产环境中的 straggler 节点或性能瓶颈。例如，当某些节点出现网络问题时，HDFS 传输阶段的上传/下载时间增长会清晰可见。同时，用户可从热力图入口查看每个 rank 的时间线分解，以全面评估系统内各项优化。图 12 则展示了某一 rank 的各阶段执行细节。

存储侧监控。在存储客户端侧，我们监控每个原子读写操作的延迟与 I/O 大小（以 I/O chunk 为粒度）。所有指标会周期性通过消息队列发送至 ClickHouse 服务进行聚合分析。当出现异常高延迟或低带宽时会触发告警，便于工程师进一步排查。在存储集群侧，我们关注整体性能指标，如元数据请求 QPS、集群级读写吞吐与容量利用率。该监控体系可有效定位慢读/慢写甚至因存储容量耗尽导致的系统不可用问题，从而提前采取措施提升系统可靠性与效率。

6 评估

在本节中，我们介绍 ByteCheckpoint 的部署与运行经验。ByteCheckpoint 构建于 DCP [57] (commit hash: 80c07df) 之上，该系统支持 FSDP [72] 的 checkpoint resharding。ByteCheckpoint 的实现约 20,000 行 Python 代码。

6.1 Checkpointing 效率

我们在不同 workload 下比较 ByteCheckpoint 与基线 [51, 57] 的 checkpointing 性能。

Workloads. 我们采用两类 transformer [61] 结构：DiT [40] 与 GPT-3 [9]，分别实现两个模型 *vDiT* 与 *tGPT*。*vDiT* 使用 FSDP 在 NVIDIA A100 80GB GPU 集群上进行视频生成 fine-tuning；*tGPT* 使用 Megatron-LM [47] 在 NVIDIA H800 80GB GPU 上进行文本生成训练。模型与并行配置见表 3：source GPU 数与 parallelism 对应用于保存与加载评估，target GPU 数与 parallelism 用于 load-time resharding 评估。所有训练机器通过 InfiniBand 互联，实验统一使用 HDFS 作为持久化存储。我们在 LFM 训练试验中集成测试模型，训练 500 步，每 100 步保存一次 checkpoint。随后在两种场景下恢复训练：GPU 数与 parallelism 不变、以及发生变化，以评估 load-time resharding 与常规加载性能。保存性能以平均额外开销衡量，包括 checkpoint stall（训练阻塞时间）与端到端保存时间（从 save API 调用到完成完整性检查）。加载性能以端到端加载时间（load API 的阻塞时长）衡量，覆盖 load-time resharding 与常规加载。我们使用平均 ETTR 衡量端到端系统性能。

遵循 GEMINI [65] 的假设：故障在每个 checkpoint 间隔（100 步）内均匀发生，据此计算平均浪费时间并得到 ETTR。计算细节见附录 C。

Baselines. 在 FSDP 实验中，我们使用 DCP (commit hash: c7338f4) 作为基线；在 Megatron-LM 实验中使用 MCP [51] (commit hash: 3fb5c51)。为公平对比，所有系统保持相同的超参数（如 batch size、context length）。两种基线均支持模型与优化器的异步 checkpointing 与 load-time resharding，但不支持 dataloader 状态。因此，GPU 状态的 I/O 性能与基线对比，同时评估 ByteCheckpoint 的完整训练状态性能。由于 DCP/MCP 原生不支持 HDFS，我们为其实现 HDFS 访问逻辑，并与 ByteCheckpoint 使用相同配置（如线程数、文件分块大小等）。

保存性能。如表 4 所示，ByteCheckpoint 在运行时显著降低 checkpoint stall，降幅在 $13.06\times$ 到 $161.50\times$ 之间，使平均 stall 时间从分钟/秒级降至亚秒级，优于 DCP 与 MCP。在端到端保存时间方面，ByteCheckpoint 平均加速 $6.05\times$ ，主要源于细粒度性能优化。加速幅度随训练规模增大而提升：从 2400 GPUs 的 $2.21\times$ 增长到 4800 GPUs 的 $8.87\times$ 。这主要归因于负载均衡机制在更大的 DP 规模下更有效。我们还观察到 FSDP workload 的阻塞时间显著降低，这是因为 DCP 在处理 FSDP irregular tensor shards 时存在较高通信与同步开销，且规模越大开销越高，使 DCP 难以胜任大规模训练。相比之下，ByteCheckpoint 的分解表示在生成 metadata 时不引入通信开销，因而效率更高。附录 D 提供了更详细的开销分解。对于 Megatron-LM 训练，我们报告了完整训练状态的 I/O 性能。

加载与 **resharding** 性能。在 parallelism 不变的加载实验中，ByteCheckpoint 相比基线提升 $1.41\times$ 到 $8.80\times$ 。异步加载 pipeline 能重叠 tensor 读取与传输，显著减少训练开始前的阻塞。对于模型与优化器状态的 resharding，ByteCheckpoint 平均加速 $3.64\times$ 。通过避免昂贵的离线 resharding 作业，ByteCheckpoint 支持不同场景下灵活高效的 checkpoint 转换。但当包含 CPU 状态（主要是 dataloader）时，端到端 load-time resharding 时间会增加，这是因为 dataloader 独有组件（如 token buffer）处理耗时较长，在 text-to-video LFM 训练中可达 20GB。此外，只有部分 worker 持有这些 dataloader 状态，导致其在保存与加载（resharding）过程中成为 straggler。我们将更高效的 dataloader 设计留作未来工作。

端到端性能。ByteCheckpoint 在平均 ETTR 上优于 DCP 与 MCP，提升幅度为 $1.16\times$ 到 $1.29\times$ 。借助 full-stack 优化，ByteCheckpoint 提升了每个 checkpoint 相关操作的执行效率，从而有效减少训练停顿。

6.2 Microbenchmarks

我们进行了多组 microbenchmark（表 5、表 6 与表 7）以展示性能提升。为评估不同模型规模，我们将 tGPT 70B 修改为 tGPT 13B 与 tGPT 30B。保存与加载 microbenchmark 使用 Megatron-LM 训练框架，resharding 使用 FSDP。

Table 3: 模型与并行配置。

Model	Hidden Size	#Heads	#Layers	#Parameters	Source #GPUs	Source Parallelism	Target #GPUs	Target Parallelism
vDiT	1664	16	48	4B	32	ZeRO-2	64	ZeRO-2
					128	ZeRO-2	64	ZeRO-2
tGPT	8192	64	80	70B	2400	TP=4, DP=75, PP=8	4800	TP=4, DP=150, PP=8
					4800	TP=4, DP=150, PP=8	2400	TP=4, DP=75, PP=8

Table 4: ByteCheckpoint、DCP 与 MCP 的 I/O 性能对比。

Model and Framework	Source #GPUs	Method	T _{Block} (s)	T _{Save} (s)	T _{Load} (s)	T _{Reshard} (s)	ETTR (%)
vDiT 4B	32	DCP	16.25	86.82	50.12	74.89	38.60
		ByteCheckpoint (GPU states)	0.54 (30.09 ×	27.47 (3.16 ×	11.69 (4.29 ×	16.01 (4.68 ×	46.22 (1.20 ×
FSDP	128	DCP	61.37	236.34	105.74	91.01	41.62
		ByteCheckpoint (GPU states)	0.38 (161.50 ×	23.74 (9.96 ×	12.01 (8.80 ×	13.64 (6.67 ×	48.92 (1.18 ×
tGPT 70B	2400	MCP	4.73	28.97	69.87	126.30	34.61
		ByteCheckpoint (GPU states)	0.39 (12.13 ×	13.11 (2.21 ×	49.48 (1.41 ×	62.10 (2.03 ×	40.18 (1.16 ×
		ByteCheckpoint (full states)	0.50	20.55	72.35	401.21	28.80
Megatron-LM	4800	MCP	4.70	76.21	123.80	64.62	31.28
		ByteCheckpoint (GPU states)	0.36 (13.06 ×	8.59 (8.87 ×	64.39 (2.00 ×	54.31 (1.19 ×	40.29 (1.29 ×
		ByteCheckpoint (full states)	0.43	25.04	83.56	115.95	34.70

Table 5: 保存优化 microbenchmark。

Workload	Parallel Config.	Optimization	Saving Time (s)
tGPT 13B	TP=2, DP=8, PP=2	No Optim.	50.26
		Async.	34.68 (1.45 ×
		Async. + WB.	20.28 (2.48 ×
		Async. + WB. + Cache.	19.97 (2.52 ×
tGPT 30B	TP=2, DP=8, PP=4	No Optim.	46.34
		Async.	25.56 (1.81 ×
		Async. + W.B.	18.83 (2.46 ×
		Async. + W.B. + Cache.	18.56 (2.50 ×

Table 6: 加载优化 microbenchmark。

Model	Parallel Config.	Optimization	Loading Time (s)
tGPT 13B	TP=2, DP=8, PP=2	No Optim.	63.48
		Async.	48.43 (1.31 ×
		Async. + Overlap.	41.38 (1.53 ×
tGPT 30B	TP=2, DP=8, PP=4	No Optim.	77.02
		Async.	74.54 (1.03 ×
		Async. + Overlap.	48.73 (1.58 ×

Table 7: Resharding 优化 microbenchmark。

Model	Parallel Config.	Optimization	Processing Time (s)
tGPT 13B	ZeRO-2 32 GPUs	All-gather + D2H.	4.12
		Decompose.	0.21 (19.80 ×
tGPT 30B	ZeRO-2 64 GPUs	All-gather + D2H.	5.84
		Decompose.	0.19 (30.50 ×

在表 5 中，我们比较了在启用与未启用 ByteCheckpoint 功能 (Sec. 4) 时的端到端保存时间。结果表明，异步保存 pipeline 相比未优化设置显著降低保存时间，tGPT 13B 与 tGPT 30B 的加速分别为 1.45× 与 1.81×。进一步启用负载均衡与 plan 缓存后，可继续提升。当对模型与优化器状态应用全部 I/O 优化时，平均保存时间从 48.3 秒降至 19.27 秒。在表 6 中，我们展示了异步加载 pipeline 与读-通信重叠技术带来的收益。对于 tGPT 13B 与 tGPT 30B，基线加载时间分别比

ByteCheckpoint 慢 1.53× 与 1.58×。

对于 load-time resharding，我们对比了 FSDP 与 ByteCheckpoint 的 irregular tensor 处理时间。如表 7 所示，ByteCheckpoint 的平均阻塞时间仅 0.20s，较 FSDP 缩短 25.15×。

7 结论

本论文介绍了 ByteCheckpoint 的设计、实现与部署，这是一套面向 LFM 开发的生产级 checkpointing 系统。ByteCheckpoint 提出统一的 checkpoint 表示与 workflow，支持高效的 load-time resharding，并兼容多种训练框架与 storage backend。系统集成 full-stack I/O 性能与可扩展性优化，并提供高效的性能监控与分析工具，支撑大规模 LFM 开发。与最先进基线 [51, 57] 相比，ByteCheckpoint 将 checkpoint stall 最多降低 161.50×，端到端 checkpoint 完成时间最多缩短 9.96×，加载 (resharding) 流程平均加速 3.88×。我们相信该系统不仅为真实 LFM 生产中的 checkpointing 系统构建提供了有价值的工程经验，也为社区未来研究提供了重要启发。

8 致谢

我们感谢 shepherd Ayush Goel 以及匿名审稿人的建设性反馈。感谢 Juncai Liu 与 Xiang Li 在大规模实验中的支持与付出。感谢 Shibiao Nong、Shuaishuai Cao、Wei Jia、Jingzhe Tang、Jun Wang、Xi Wang、Sa Wang 以及其他贡献者对系统的完善与功能扩展。感谢 Liyang Zhao 和 ByteDance 的 HDFS 团队提供的支持。本工作部分由 ByteDance Research Collaboration Project 以及香港 RGC 项目 (HKU 17204423 与 C7004-22G (CRF)) 资助。

Table 8: ByteCheckpoint 在大规模 LFM 训练中的 I/O 性能。

Model and Framework	#GPUs	Parallelism	T _{Block} (s)	T _{Save} (s)	T _{Load} (s)
Vision Transformer 7B FSDP	1488	ZeRO-2	0.34	20.13	265.73
Text Transformer 405B Megatron-LM	8960	TP=8, DP=70, PP=16	0.59	51.06	129.49

References

- [1] HDFS Federation Solution: NameNodeProxy. <https://github.com/bytedance/nnproxy>, 2016.
- [2] Safetensors. <https://huggingface.co/docs/safetensors/index>, 2024.
- [3] TensorStore. <https://google.github.io/tensorstore/>, 2024.
- [4] Zarr. <https://zarr.dev/>, 2024.
- [5] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [6] Wei An, Xiao Bi, Guanting Chen, Shanhuang Chen, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Wenjun Gao, Kang Guan, et al. Fire-Flyer AI-HPC: A cost-effective software-hardware co-design for deep learning. *arXiv preprint arXiv:2408.14158*, 2024.
- [7] Sanjith Athlur, Nitika Saran, Muthian Sivathanu, Ramachandran Ramjee, and Nipun Kwatra. Varuna: scalable, low-cost training of massive deep learning models. In *Proceedings of the Seventeenth European Conference on Computer Systems*, pages 472–487, 2022.
- [8] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [9] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.
- [10] Trishul Chilimbi, Yutaka Suzue, Johnson Apacible, and Karthik Kalyanaraman. Project adam: Building an efficient and scalable deep learning training system. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 571–582, 2014.
- [11] et al. DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024.
- [12] Jiangfei Duan, Ziang Song, Xupeng Miao, Xiaoli Xi, Dahua Lin, Harry Xu, Minjia Zhang, and Zhihao Jia. Parcae: Proactive, {Liveput-Optimized}{DNN} training on preemptible instances. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 1121–1139, 2024.
- [13] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [14] Assaf Eisenman, Kiran Kumar Matam, Steven Ingram, Dheevatsa Mudigere, Raghuraman Krishnamoorthi, Krishnakumar Nair, Misha Smelyanskiy, and Murali Annavaram. Check-N-Run: A checkpointing system for training deep learning recommendation models. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 929–943, 2022.
- [15] Apache Software Foundation. Hadoop Distributed File System. <https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>, 2024.
- [16] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. {ServerlessLLM}:{Low-Latency} serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 135–153, 2024.
- [17] gRPC Team. grpc: A high performance, open source universal rpc framework get started! <https://grpc.io/>, 2024.
- [18] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

- [19] Tanmaey Gupta, Sanjeev Krishnan, Rituraj Kumar, Abhishek Vijeev, Bhargav Gulavani, Nipun Kwatra, Ramachandran Ramjee, and Muthian Sivathanu. Just-In-Time Checkpointing: Low cost error recovery from deep learning training failures. In Proceedings of the Nineteenth European Conference on Computer Systems, pages 1110–1125, 2024.
- [20] Tao He, Xue Li, Zhibin Wang, Kun Qian, Jingbo Xu, Wenyan Yu, and Jingren Zhou. Unicorn: Economizing self-healing llm training at scale. arXiv preprint arXiv:2401.00134, 2023.
- [21] Frank Herold, Sven Breuner, and Jan Heichler. An introduction to BeeGFS. ThinkParQ, Kaiserslautern, Germany, Tech. Rep, 2014.
- [22] Qinghao Hu, Zhisheng Ye, Zerui Wang, Guoteng Wang, Meng Zhang, Qiaoling Chen, Peng Sun, Dahua Lin, Xiaolin Wang, Yingwei Luo, et al. Characterization of large language model development in the datacenter. In 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24), pages 709–729, 2024.
- [23] Drew A Hudson and Christopher D Manning. GQA: A new dataset for real-world visual reasoning and compositional question answering. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 6700–6709, 2019.
- [24] Insu Jang, Zhenning Yang, Zhen Zhang, Xin Jin, and Mosharaf Chowdhury. Ooblock: Resilient distributed training of large models using pipeline templates. In Proceedings of the 29th Symposium on Operating Systems Principles, pages 382–395, 2023.
- [25] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, et al. MegaScale: Scaling large language model training to more than 10,000 GPUs. In 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24), pages 745–760, 2024.
- [26] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980, 2014.
- [27] Apostolos Kokolis, Michael Kuchnik, John Hoffman, Adithya Kumar, Parth Malani, Faye Ma, Zachary DeVito, Shubho Sengupta, Kalyan Saladi, and Carole-Jean Wu. Revisiting reliability in large-scale machine learning research clusters. arXiv preprint arXiv:2410.21680, 2024.
- [28] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models. Proceedings of Machine Learning and Systems, 5:341–353, 2023.
- [29] Jiamin Li, Hong Xu, Yibo Zhu, Zherui Liu, Chuanxiong Guo, and Cong Wang. Lyra: Elastic scheduling for deep learning clusters. In Proceedings of the Eighteenth European Conference on Computer Systems, pages 835–850, 2023.
- [30] Mingzhen Li, Wencong Xiao, Hailong Yang, Biao Sun, Hanyu Zhao, Shiru Ren, Zhongzhi Luan, Xianyan Jia, Yi Liu, Yong Li, et al. Easyscale: Elastic training with consistent accuracy and improved utilization on gpus. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1–14, 2023.
- [31] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. Pytorch distributed: Experiences on accelerating data parallel training. arXiv preprint arXiv:2006.15704, 2020.
- [32] Xinyu Lian, Sam Ade Jacobs, Lev Kurilenko, Masahiro Tanaka, Stas Bekman, Olatunji Ruwase, and Minjia Zhang. Universal checkpointing: Efficient and flexible checkpointing for large scale distributed training. arXiv preprint arXiv:2406.18820, 2024.
- [33] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437, 2024.
- [34] Avinash Maurya, Robert Underwood, M Mustafa Rafique, Franck Cappello, and Bogdan Nicolae. DataStates-LLM: Lazy asynchronous checkpointing for large language models. arXiv preprint arXiv:2406.10707, 2024.
- [35] Jayashree Mohan, Amar Phanishayee, and Vijay Chidambaram. CheckFreq: Frequent, fine-Grained DNN checkpointing. In 19th USENIX Conference on File and Storage Technologies (FAST 21), pages 203–216, 2021.
- [36] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary,

- Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. Efficient large-scale language model training on gpu clusters using Megatron-LM. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1–15, 2021.
- [37] Bogdan Nicolae, Jiali Li, Justin M Wozniak, George Bosilca, Matthieu Dorier, and Franck Cappello. Deepfreeze: Towards scalable asynchronous checkpointing of deep learning models. In 2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID), pages 172–181. IEEE, 2020.
- [38] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. Advances in neural information processing systems, 35:27730–27744, 2022.
- [39] Satadru Pan, Theano Stavrinou, Yunqiao Zhang, Atul Sikaria, Pavel Zakharov, Abhinav Sharma, Mike Shuey, Richard Wareing, Monika Gangapuram, Guanglei Cao, et al. Facebook’s tectonic filesystem: Efficiency from exascale. In 19th USENIX Conference on File and Storage Technologies (FAST 21), pages 217–231, 2021.
- [40] William Peebles and Saining Xie. Scalable diffusion models with transformers. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 4195–4205, 2023.
- [41] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. Advances in Neural Information Processing Systems, 36, 2024.
- [42] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZERO: Memory optimizations toward training trillion parameter models. In SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1–16. IEEE, 2020.
- [43] Machel Reid, Nikolay Savinov, Denis Teplyashin, Dmitry Lepikhin, Timothy Lillicrap, Jean-baptiste Alayrac, Radu Soricut, Angeliki Lazaridou, Orhan Firat, Julian Schrittwieser, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv preprint arXiv:2403.05530, 2024.
- [44] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 10684–10695, 2022.
- [45] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.
- [46] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. arXiv preprint arXiv:2409.19256, 2024.
- [47] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. arXiv preprint arXiv:1909.08053, 2019.
- [48] Character AI Team. Character.ai. <https://character.ai/>, 2024.
- [49] DeepSeek Team. Fire-flyer file system. <https://github.com/deepseek-ai/3FS>, 2025.
- [50] Github Team. The world’s most widely adopted ai developer tool. <https://github.com/features/copilot/>, 2024.
- [51] Megatron Team. Dist checkpointing package. https://docs.nvidia.com/megatron-core/developer-guide/latest/api-guide/dist_checkpointing.html, 2024.
- [52] Midjourney Team. Midjourney. <https://www.midjourney.com/home>, 2023.
- [53] NVIDIA NCCL Team. Nvidia collective communications library (nccl). <https://developer.nvidia.com/nccl>, 2024.
- [54] OpenAI Team. Creating video from text. <https://openai.com/index/sora/>, 2024.
- [55] OpenAI Team. Introducing ChatGPT. <https://openai.com/index/chatgpt/>, 2024.
- [56] OpenAI Team. Introducing OpenAI o1. <https://openai.com/o1/>, 2024.

- [57] PyTorch Team. Getting started with Distributed Checkpoint (DCP). https://pytorch.org/tutorials/recipes/distributed_checkpoint_recipe.html, 2023.
- [58] Suno Team. Bark is a transformer-based text-to-audio model created by suno. <https://github.com/suno-ai/bark>, 2023.
- [59] Zarr Team. Zarr: chunked, compressed, n-dimensional arrays. <https://zarr.dev/>, 2024.
- [60] John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. Bamboo: Making preemptible instances resilient for affordable training of large DNNs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 497–513, Boston, MA, April 2023. USENIX Association.
- [61] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- [62] veScale Team. veScale: A PyTorch Native LLM Training Framework. <https://github.com/volcengine/veScale>, 2024.
- [63] Marcel Wagenländer, Guo Li, Bo Zhao, Luo Mai, and Peter Pietzuch. Tenplex: Dynamic parallelism for deep learning using parallelizable tensor collections. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 195–210, 2024.
- [64] Yuxin Wang, Shaohuai Shi, Xin He, Zhenheng Tang, Xinglin Pan, Yang Zheng, Xiaoyu Wu, Amelie Chi Zhou, Bingsheng He, and Xiaowen Chu. Reliable and efficient in-memory fault tolerance of large language model pretraining. *arXiv preprint arXiv:2310.12670*, 2023.
- [65] Zhuang Wang, Zhen Jia, Shuai Zheng, Zhen Zhang, Xinwei Fu, TS Eugene Ng, and Yida Wang. Gemini: Fast failure recovery in distributed training with in-memory checkpoints. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 364–381, 2023.
- [66] Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021.
- [67] Wencong Xiao, Shiru Ren, Yong Li, Yang Zhang, Pengyang Hou, Zhi Li, Yihui Feng, Wei Lin, and Yangqing Jia. {AntMan}: Dynamic scaling on {GPU} clusters for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 533–548, 2020.
- [68] Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang, Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu, Wendi Zheng, Xiao Xia, et al. GLM-130b: An open bilingual pre-trained model. *arXiv preprint arXiv:2210.02414*, 2022.
- [69] Chi Zhang, Guangming Sheng, Siyao Liu, Jiahao Li, Ziyuan Feng, Zherui Liu, Xin Liu, Xiaoying Jia, Yanghua Peng, Haibin Lin, et al. A framework for training large language models for code generation via proximal policy optimization.
- [70] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuhui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. OPT: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.
- [71] Juntao Zhao, Borui Wan, Yanghua Peng, Haibin Lin, Yibo Zhu, and Chuan Wu. QSync: Quantization-minimized synchronous distributed training across hybrid devices. *arXiv preprint arXiv:2407.02327*, 2024.
- [72] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. Pytorch FSDP: Experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277*, 2023.
- [73] Yuchen Zhong, Guangming Sheng, Juncheng Liu, Jinhui Yuan, and Chuan Wu. Swift: Expedited failure recovery for large-scale dnn training. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, pages 447–449, 2023.
- [74] Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y Wu, Yukun Li, Huazuo Gao, Shirong Ma, et al. DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence. *arXiv preprint arXiv:2406.11931*, 2024.

```

Offline Script for MoE Checkpoint Resharding
python3 reshard_megatron_ckpt/reshard_moe_v2_3.py --prev_tp=8 --prev_dp=100 --prev_pp=12 \
--target_tp=4 --target_dp=200 --target_pp=8 --gate_fp32=True --sp_attn=True

Offline Script for GPT Checkpoint Resharding
python3 reshard_megatron_ckpt/reshard_gpt_v4_8.py --prev_tp=8 --prev_dp=100 --prev_pp=6 \
--prev_vp=10 --target_tp=8 --target_dp=200 --target_pp=6 --target_vp=10 --sp_attn=False

```

Figure 13: 离线 resharding 脚本运行示例。

A 维护 Resharding 脚本的更多背景

图 13 展示了使用定制脚本进行离线 resharding 的示例。定制离线脚本缺乏扩展性且开发成本高。对于 GPU 状态 resharding，脚本必须覆盖 LFM 与其优化器的所有组件，并适配不同 parallelism 策略下各组件的差异行为。例如，在 Tensor Parallelism (TP) 下，注意力与 MLP 模块中的两个 GEMM 操作沿不同维度分片，而 LayerNorm [8] 等算子则在 GPU 间复制。当采用 3D 混合并行 [36] 时，分布式优化器中的 TP 分片张量会先被扁平化并合并，再按指定的 Data Parallelism (DP) 度进行分片。离线脚本必须实现与模型（优化器）组件和 parallelism 组合强耦合的 resharding 逻辑。此外，GQA [23] 与 MLA [74] 等算法优化会改变某些算子的 tensor 布局（如注意力块中的 QKV 投影 GEMM），也需要相应的 resharding 支持。为应对生产环境中的多种情况，我们维护了多个版本的 resharding 脚本，最大的一份甚至包含 3193 行 Python 代码。这种复杂性带来了显著的开发与维护负担。

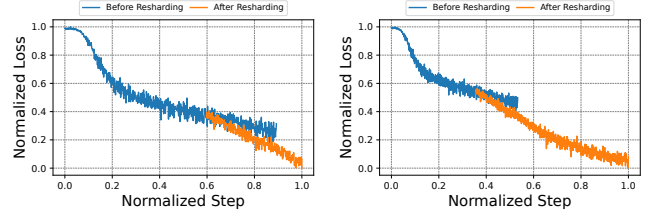
B 高效的完整性保障

一个完整 checkpoint 由多个 worker 写入的多个文件组成，任一 worker 失败都可能导致 checkpoint 不完整。因此，需要 barrier 机制在所有训练 worker 间实现保存/加载操作的原子性。

诸如 Megatron-LM [47] 的 checkpointing 模块使用 `torch.distributed` 的 `barrier` 来做完整性检查，该方式会同步所有 worker，确保保存/加载完成，但也带来训练阻塞。我们观察到，当训练规模达到约 10,000 GPUs 时，每次 barrier 会造成约 20 秒的 stall。为降低该开销，我们使用前述 gRPC + 树形分层通信拓扑重实现 `barrier`，并将完整性检查异步化，从而消除阻塞时间。同时，ByteCheckpoint 的 I/O worker 内置上传/下载重试机制，并集成失败日志，记录未完成 checkpoint 任务的 worker 在保存/加载 pipeline 中的失败阶段，便于进一步诊断。

C 平均 ETTR 计算

训练中的故障会引入进度损失与最近一次 checkpoint 加载开销。假设故障在每个 checkpoint 间隔内均匀分布 [65]，最优情况是故障发生在一次端到端保存刚完成之后，最差情况是发生在保存即将完成之前。给定每步训练时间 T_{iter} 、checkpoint 间隔 N 、端到端保存时



(a) DP resharding: TP=1, DP=4, PP=4 → TP=1, DP=4, PP=4 → TP=2, DP=8, PP=4。
(b) 混合 resharding: TP=1, DP=4, PP=4 → TP=2, DP=8, PP=2。

Figure 14: Resharding 正确性验证。

间 T_{save} 与加载 (resharding) 时间 T_{load} ，平均浪费时间 T_{wasted} 为：

$$T_{wasted} = T_{save} + T_{load} + \frac{N \cdot T_{iter}}{2} \quad (1)$$

因此平均 ETTR 为：

$$ETTR = 1 - \frac{T_{wasted}}{T_{save} + T_{load} + N \cdot T_{iter}} \quad (2)$$

表 4 中的端到端 ETTR 结果为标准加载与 resharding 设置下的平均值。

D Checkpointing 开销分解

我们将 rank 0 的端到端 checkpoint 保存时间 (T_{save}) 拆解为多个阶段，并分析各部分开销。结果如表 9 所示，其中 T_{Plan}^{First} 表示首次规划成本， T_{Plan}^{Cache} 表示后续使用 plan 与 metadata 缓存时的开销。其他阶段开销取多个 checkpoint 步的平均值。我们发现，随着训练规模增大，规划的通信开销显著上升并导致明显 stall。得益于缓存策略 (Sec. 4.1)，规划成本可转化为每次（恢复后的）训练会话的一次性支出。此外，固定页 CPU 内存池使 D2H 阻塞几乎可以忽略；异步 engine pipeline 将序列化、共享内存 dump 与 HDFS 上传阶段重叠，从而缩短端到端时间。负载均衡机制也提升了 DP 组内的并行上传能力，在更大规模下带来更多收益（例如，4800 GPUs 的模型状态上传速度比 2400 GPUs 快 3.03×）。

E 更多 Resharding 正确性实验

在 DP 与混合 resharding 的正确性验证中，归一化 loss 曲线如图 14 所示。由于在 DP 与混合 resharding 中我们同时增大了全局 batch size，resharding 后的 loss 下降更快。

我们进一步验证 ByteCheckpoint 可以对 dataloader 状态实现 bitwise 正确的恢复。由于 RNG 状态固定，正确恢复应产生相同的数据采样轨迹。因此我们使用归一化样本长度曲线 (图 15) 进行评估。结果显示，重启前后的归一化样本长度曲线一致。

Table 9: rank 0 上 checkpoint 保存过程的详细开销分解。

Model and Framework	Source #GPUs	State	$T_{\text{Plan}}^{\text{First}}$ (s)	$T_{\text{Plan}}^{\text{Cache}}$ (s)	T_{D2H} (s)	$T_{\text{Serialize}}$ (s)	T_{Dump} (s)	T_{Upload} (s)
vDiT 4B	32	Model	0.05	0.00	0.15	0.52	0.50	11.04
		Optimizer	0.65	0.00	0.37	0.93	1.03	14.60
	128	Model	0.35	0.00	0.06	0.29	0.12	10.57
		Optimizer	0.89	0.00	0.06	0.29	0.26	9.85
tGPT 70B	2400	Model	8.39	0.00	0.08	1.93	0.48	6.67
		Optimizer	5.02	0.00	0.17	2.00	2.23	2.30
	4800	Model	17.09	0.00	0.08	1.95	0.47	2.20
		Optimizer	6.46	0.00	0.03	1.69	0.35	1.67

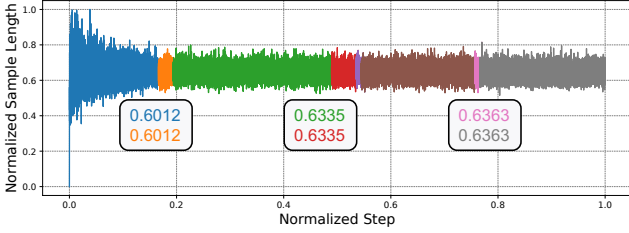


Figure 15: 多次训练重启下 dataloader 的归一化样本长度曲线。

F 相关工作

Checkpointing 框架。一些工业实践专注于深度学习 checkpointing 系统。在 DCP [57] 之前, PyTorch 仅提供 `torch.save` 与 `torch.load` 用于本地 checkpoint 管理, 缺乏原生 resharding 支持。DCP [57] 为 FSDP 引入了 resharding 能力, 但不支持 TP、PP 等混合并行策略。DeepSpeed-UCP [32] 提供统一的 DeepSpeed checkpoint 格式, 并通过离线脚本实现 resharding。Megatron MCP [51] 复用 DCP [57] 的 workflow, 并扩展到 Zarr [59] 等存储格式。这些框架对不同并行策略与训练框架的支持均有限, 且在大规模训练中 I/O 性能与可扩展性不足。

Checkpointing 优化。一些工作 [10, 37] 从不同角度降低 checkpointing 成本。Check-N-Run [14] 面向推荐模型, 采用差分 checkpointing 仅存储修改部分, 并结合量化降低 checkpoint 大小。CheckFreq [35] 将 snapshot 与保存与计算进行流水线化, 减少 checkpoint stall, 并引入在线算法调优 checkpoint 频率。Gemini [65] 倡导 in-memory checkpointing 与跨机器备份以加速恢复, 并将 checkpoint 通信与训练流量交错, 实现每次迭代的高频 checkpoint。JIT-Checkpointing [19] 采用 just-in-time checkpointing 进行低成本错误恢复。在工业用例中, 仍需将 checkpoints 保存到独立持久化存储, 以支持 auto-evaluation [22]、超参调优与模型调试等任务。ServerlessLLM [16] 关注 serverless 推理场景下的加载优化, 提出基于 chunk 的多级加载 pipeline 以加速 checkpoint 加载。与这些方案不同, ByteCheckpoint 同时强调优化 I/O 性能与灵活的 load-time checkpoint resharding, 面向通用 LFM 开发场景。

Checkpoint 表示。PyTorch 的 `torch.save/load` 基于 `pickle` 进行序列化/反序列化, 该格式缺乏全局形状等关键 shard metadata, 无法支持自动 resharding。DCP [57] 通过将 metadata 与 tensor 数据分离来解决这一问题, metadata 包含全局形状与偏移信息, 从而支持自动 resharding。Tenplex [63] 提出 Parallelizable Tensor Collection (PTC) 以灵活表示与转换不同 parallelism 配置下的 tensor 状态。ByteCheckpoint 采用 DCP 的表示并扩展以处理 irregular tensor 分片, 从而构建 PyTorch-native 的 checkpointing 系统。数组式存储系统如 Zarr [4] 与 TensorStore [3] 可将 tensor 保存为独立数组, 支持并发读写; MCP [51] 通过 Zarr 实现分布式 checkpointing。Safetensors [2] 提供安全高效的 tensor 存储与加载格式。为增强与 Hugging Face 生态兼容性, ByteCheckpoint 支持导出 Safetensors 格式的 checkpoints。

LFM 的存储系统。大规模 checkpointing 需要稳健的存储系统。LLaMA 3.1 报告 [13] 指出 Tectonic [39] (Meta 的通用文件系统) 是 pre-training 期间存储 checkpoints 的骨干, 其核心挑战之一是高频 checkpoint 写入。类似地, DeepSeek 开发的 AI-HPC 系统 FireFlyer [6] (用于 LFM 训练 [11, 18, 33]) 采用自研 3FS 分布式文件系统 [49], 与 BeeGFS [21] 等并行文件系统类似, 用于 checkpoint 存储。

弹性训练系统。一些正交研究致力于提升 DL 训练任务的弹性 [7, 12, 24, 30, 60, 73]。例如, Varuna 引入 *job morphing* 以在不改变超参的情况下调整 pipeline 与 data parallelism 度; Bamboo [60] 将跨阶段冗余计算融入 pipeline, 以支持 spot 实例上的弹性训练; Oobleck [24] 通过预定义模板进行 pipeline 实例化, 以容忍不同 pipeline 的并发故障。这些工作通常仅覆盖特定并行策略 (如不含 ZeRO 的 DP), 而 ByteCheckpoint 不假设具体 parallelism, 可支持真实生产场景。采用我们的 checkpoint 表示可进一步增强弹性训练系统的状态管理能力。